

TỐI ƯU HOÁ VẬN HÀNH MÁY BAY PHUN THUỐC TRỪ SÂU

Tổng quan

Từ các dữ kiện được cho trong đề bài và các thông tin thu thập được qua mạng Internet, chúng tôi xây dựng hàm tiêu hao pin của Drone dựa trên tài liệu kỹ thuật của nhà sản xuất và đánh giá thực tế về các nhân tố tiêu thụ lớn nhất. Trong bước này, chúng tôi đồng thời đưa ra một số các giả sử hợp lý liên quan tới điều kiện hoạt động và mục đích sử dụng của Drone.

Sử dụng hàm tiêu thụ năng lượng, chúng tôi xây dựng chương trình mô phỏng hoạt động của Drone bao gồm tất cả các biến số của hàm. Từ đó, chúng tôi xác định một số các chiến thuật thiết kế đường bay hợp lý cho các thửa ruộng đã cho, ví dụ như theo từng hàng, hoặc các vòng tròn đồng tâm, và cuối cùng đi đến xác định chiến thuật hợp lý cho trường hợp tổng quát.

Chúng tôi kết luận được công thức để tính lượng tiêu hao pin của drone, đưa ra được 2 chiến thuật di chuyển cho mỗi mảnh ruộng và nhận thấy rằng việc di chuyển theo các vòng đồng tâm cho hiệu quả cao hơn.

Mục lục

1	Giới thiệu	3
2	Mô hình bài toán	3
3	Các giả sử	4
4	Công thức và Thông số	6
4.1	Hàm tiêu hao pin	6
4.2	Độ phủ và tốc độ bay	7
4.3	Nhận xét về các mảnh ruộng	7
5	Chiến thuật di chuyển	8
5.1	Mảnh ruộng 1	8
5.1.1	Chiến thuật 1: Đi theo hàng	8
5.1.2	Chiến thuật 2: Vòng đồng tâm	9
5.2	Mảnh ruộng 2	10
5.2.1	Chiến thuật 1: Chia để trị + Đi theo hàng	10
5.2.2	Chiến thuật 2: Vòng đồng tâm	12
5.3	Mảnh ruộng 3	13
5.3.1	Chiến thuật 1: Đi theo hàng	13
5.3.2	Chiến thuật 2: Chia để trị + Vòng đồng tâm	14
5.4	Tổng kết	16
6	Phụ lục	17
6.1	Chương trình mô phỏng	17
6.2	Tài liệu tham khảo	26

1 Giới thiệu

Một trong những điểm cốt lõi của cuộc cách mạng công nghiệp lần thứ 4 là IoT (Internet of things - Internet vạn vật). Không chỉ trong những ngành công nghiệp, IoT cũng tạo ra những thay đổi nhất định đối với nông nghiệp. Một trong những ví dụ điển hình nhất cho sự ảnh hưởng này là việc ứng dụng drone để tự động hoá quy trình sản xuất và canh tác.

Phun thuốc trừ sâu là một hoạt động thiết yếu đối với sự phát triển của cây trồng. Trước sự phát triển của IoT, việc phun thuốc thủ công cần rất nhiều thời gian và công sức của người nông dân để thực hiện. Không chỉ vậy, việc phun thuốc thủ công còn gây hại tới sức khoẻ người nông dân, và vẫn có sự thiếu hiệu quả, ví dụ như sử dụng quá liều thuốc, hay sử dụng loại thuốc không phù hợp với cây trồng. Sự tiến bộ của khoa học cho phép drone thực hiện các công việc này với năng suất lớn hơn nhiều lần, đồng thời phát huy tối đa hiệu quả của thuốc.

Bên cạnh phần cứng và đặc điểm kỹ thuật, thuật toán và quy trình hoạt động cũng có ảnh hưởng lớn tới hiệu suất hoạt động của drone. Ví dụ, một thuật toán tối ưu có thể xác định chính xác thời điểm drone phun hết lượng thuốc trừ sâu, và lên kế hoạch để quãng đường quay lại điểm tập kết để nạp lại là tối thiểu.

Trong bản báo cáo này, chúng tôi đưa ra mô hình hoạt động của drone cho các mảnh ruộng được cung cấp, nhằm mục đích tối ưu thời gian thực hiện công việc và năng lượng sử dụng, hướng đến giảm thiểu tối đa chi phí hoạt động.

2 Mô hình bài toán

Sau khi nghiên cứu các ví dụ thực tế, nhóm chúng tôi nhận thấy những tham số sau đây có ảnh hưởng đáng kể tới mô hình thuật toán:

1. Drone

- Trọng lượng không tải
- Tải trọng tối đa
- Dung lượng pin
- Tổng công suất các hệ thống
- Tốc độ di chuyển tối đa
- Tốc độ phun tối đa

- Tầm phun thuốc

2. Mảnh ruộng mục tiêu

- Kích thước và hình dạng
- Điểm tập kết

3 Các giả sử

Trên thực tế, có rất nhiều yếu tố ảnh hưởng tới mô hình hoạt động của drone. Tuy nhiên, phần lớn các yếu tố này không có tác động quá lớn tới đầu ra của mô hình và tương đối phức tạp. Sau đây, chúng tôi xin liệt kê một số giả sử về điều kiện hoạt động của drone.

1. Giống cây

Chúng tôi lựa chọn **lúa** là giống cây mục tiêu, do sự phổ biến tại Việt Nam và môi trường sống thích hợp với việc sử dụng drone.

2. Điều kiện hoạt động

Chúng tôi giả sử mảnh ruộng có độ cao là **0 mét so với mực nước biển** do hai vùng lúa chính của Việt Nam là Đồng bằng Sông Cửu Long và Đồng bằng Bắc Bộ có độ cao từ 0 tới 12 mét, không đáng kể đối với hoạt động của drone.

Chúng tôi giả sử trong suốt quá trình vận hành, **nhiệt độ và gió của môi trường hoạt động nằm trong ngưỡng cho phép của nhà sản xuất**. Giả sử này có được là do:

- Khí hậu Việt Nam không quá khắc nghiệt so với tiêu chuẩn hoạt động của Drone.
- Số khung giờ mà nhiệt độ và gió gây ảnh hưởng đáng kể là không nhiều.
- Trong trường hợp các điều kiện môi trường vượt quá ngưỡng cho phép, người sử dụng có thể lựa chọn thời điểm khác trong ngày.

Chúng tôi giả sử trên mảnh ruộng **không có vật cản và nhiễu động sóng điện từ**, do đây là các yếu tố không kiểm soát được, và không có ảnh hưởng lớn tới mô hình.

3. Thuốc phun

Chúng tôi giả sử thuốc phun có khối lượng riêng **1kg/L** do thành phần chính của dung dịch thuốc trừ sâu là nước máy.

Chúng tôi giả sử giống cây trồng đã chọn cần nồng độ phun là **500L/ha** (đã pha loãng) do đây là nồng độ của các loại thuốc trừ sâu phổ biến trên thị trường.

4. Drone

Chúng tôi không tính tới thời gian cất cánh, tăng tốc, chuyển hướng và thay pin.

Chúng tôi giả sử tất cả các thông số trên lý thuyết được cung cấp bởi nhà sản xuất là chính xác tuyệt đối qua tất cả các lần chạy. Nói cách khác, không có sự hao hụt về hiệu suất sau mỗi lần chạy.

Chúng tôi giả sử khi lượng thuốc trừ sâu trên drone cạn kiệt, máy bơm sẽ ngừng hoạt động. Khi đó, máy bơm sẽ không tiêu hao năng lượng.

4 Công thức và Thông số

4.1 Hàm tiêu hao pin

Để có thể xây dựng được mô hình, chúng chúng tôi cần ước tính được tốc độ tiêu hao pin của drone. Qua nghiên cứu các ví dụ thực tế và tài liệu kỹ thuật của nhà sản xuất drone, chúng tôi đưa ra công thức như sau:

$$CP = C_m + C_b + v^2 * C_r + C_p * f_p + W_t * C_t$$

Trong đó:

- CP : Chi phí trong một đơn vị thời gian (W)
- C_m : Chi phí cố định chạy hệ thống điện (W)
- C_b : Chi phí duy trì bay trên không (W)
- v : Vận tốc di chuyển (m/s)
- C_r : Hằng số lực cản không khí của drone ($W * s^2/m^2$)
- C_p : Chi phí bơm mỗi đơn vị thể tích thuốc (Wh/L)
- f_p : Tốc độ bơm (L/h)
- W_t : Khối lượng thuốc trừ sâu còn lại (kg)
- C_t : Chi phí nâng mỗi đơn vị khối lượng thuốc (W/kg)

Sau khi nghiên cứu các tài liệu kỹ thuật, chúng tôi nhận thấy hiệu quả hoạt động của drone đối với các giả sử trên chủ yếu được quyết định bởi tốc độ của bơm. Do vậy, chúng tôi lựa chọn mẫu drone DJI Agras T40 vì đây là mẫu drone có tốc độ phun cao nhất.

Do thiết kế của mẫu drone này, lượng tăng tiêu hao pin do tác dụng của lực cản không khí là không đáng kể. Vì vậy, chúng tôi coi tham số này là hằng số.

Như vậy, đối với mẫu drone **DJI Agras T40**, hàm tiêu hao pin có thể được viết dưới dạng như sau:

$$CP = C + C_p * f_p + W_t * C_t$$

Trong công thức này, chúng tôi gọi tổng các hằng số là C .

Dựa vào tài liệu được cung cấp của nhà sản xuất, chúng tôi có bảng dữ liệu sau đây:

STT	Thời gian bay	Tải trọng ban đầu	Tải trọng kết thúc
1	18 phút	50kg	50kg
2	7 phút	90kg	50kg
3	6 phút	101kg	50kg

Sử dụng bảng dữ liệu này, chúng tôi tính được các hằng số có giá trị như sau:

- $C = 11000W$
- $C_p = 0.42Wh/L$
- $C_T = 857.14W/kg$

4.2 Độ phủ và tốc độ bay

Theo thông số được cung cấp bởi nhà sản xuất, mẫu drone DJI Agras T40 có tầm phun là 10m và tốc độ tối đa là 7m/s.

4.3 Nhận xét về các mảnh ruộng

Trước tiên, chúng tôi có một số nhận xét cơ bản như sau:

- Khi đi qua phần ruộng chưa đủ nồng độ thuốc trừ sâu, chúng tôi cho bơm hoạt động với **công suất tối đa**, do tốc độ phun là điểm nghẽn của mô hình.
- Khi đi qua phần ruộng đã đủ thuốc trừ sâu, chúng tôi **không sử dụng bơm**, đồng thời cho drone di chuyển với tốc độ **tối đa** để tối thiểu hóa thời gian chết.
- Mô hình của chúng tôi sẽ dựng các đường đi phủ kín toàn bộ ruộng. Khi drone còn vừa đủ pin để quay về sạc, thiết bị sẽ ngay lập tức đổi lộ trình về sạc ở điểm tập kết rồi quay lại và tiếp tục quá trình phun.

5 Chiến thuật di chuyển

Chúng tôi điều chỉnh tốc độ và tầm phun sao cho nồng độ phun là **75L/ha**. Như vậy drone cần đi theo lộ trình đưa ra khoảng **7 lần** để đạt được nồng độ thuốc yêu cầu.

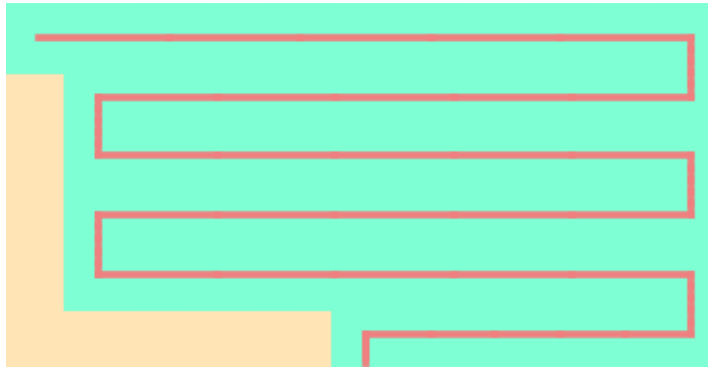
Trong những hình vẽ tiếp theo, các điểm màu vàng là các vùng chưa được đi tới; các điểm màu xanh lá là các vùng đã được phun thuốc; các điểm màu đỏ là đường đi của drone. Hình ảnh toàn bộ hoạt động của drone có trong đường dẫn tới trang Github ở cuối bài.

5.1 Mảnh ruộng 1

Đây là một mảnh ruộng có dạng hình chữ nhật có kích thước bé, với điểm bắt đầu nằm chính giữa cạnh dưới.

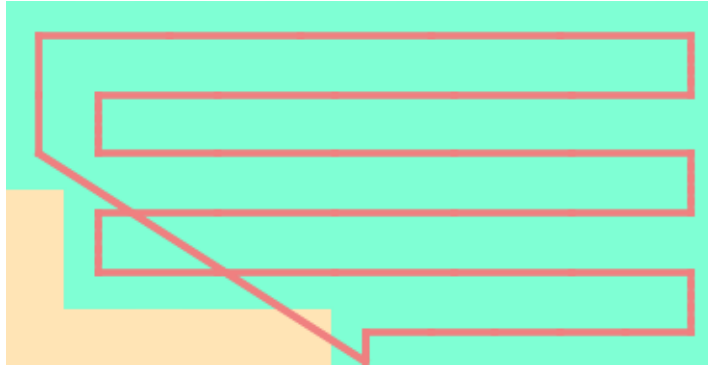
5.1.1 Chiến thuật 1: Đi theo hàng

Một chiến thuật đơn giản cho mảnh ruộng cụ thể này là đi theo từng hàng ngang. Ngoài ra, chúng tôi sẽ để lại cột cuối cùng cho việc đi về.



Hình 5.1: Drone đi theo từng hàng

Sau khi đi thêm một đoạn, drone hết thuốc trừ sâu và cần quay về để lấy thêm.

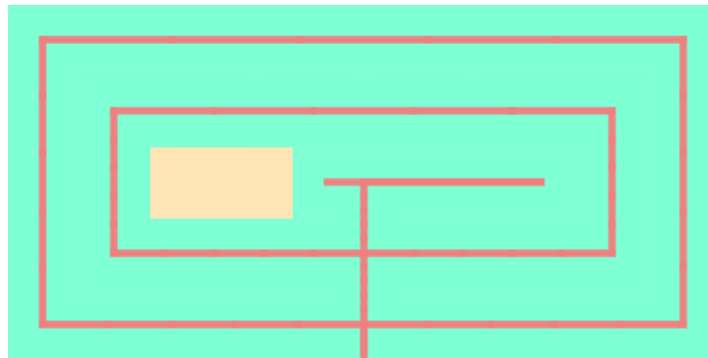


Hình 5.2: Drone quay về để lấy thêm thuốc

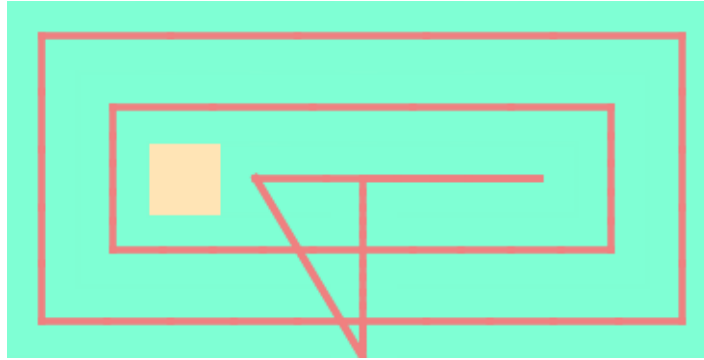
Như chúng tôi thấy, chúng tôi cần một lần quay trở về để lấy thêm thuốc. Chiến thuật này cần **1520.83 giây** để hoàn thành công việc và lượng pin sử dụng là **12650.46 Wh**.

5.1.2 Chiến thuật 2: Vòng đồng tâm

Thuật toán khác cho drone đi một vòng quanh các cạnh của thửa ruộng, và sau khi đi hết, thửa ruộng sẽ có hình chữ nhật có các cạnh bé hơn, và khi đó chúng tôi sẽ lặp lại thuật toán này.

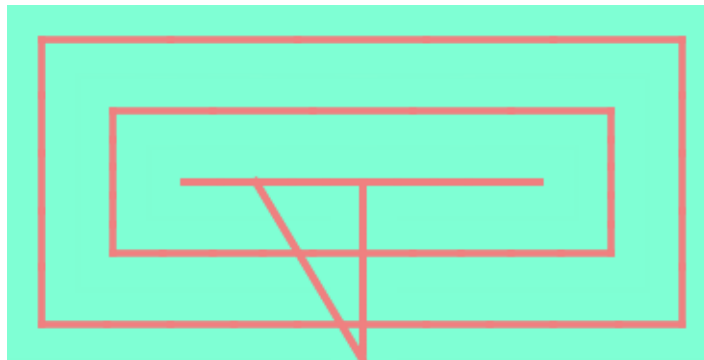


Hình 5.3: Drone trước khi phải đi về



Hình 5.4: Drone đi về để lấy thêm thuốc trừ sâu

Với thuật toán này, drone cũng chỉ cần một lần đi về. Khi chạy thuật toán này trên chương trình mô phỏng, drone mất **1495.54 giây** để thực hiện và tiêu thụ **12419.31 Wh**.



Hình 5.5: Drone thành công phun toàn bộ ruộng

Như vậy, bằng việc thay đổi tuyến đường, chúng tôi có thể tiết kiệm được 30 giây và tới 200 Wh cho một thửa ruộng diện tích chỉ nửa héc-ta.

Chúng tôi rút ra được kết luận: Đối với mảnh ruộng 1, **chiến thuật vòng đồng tâm** cần sử dụng lượng điện và thời gian ít hơn so với **chiến thuật đi theo hàng**.

5.2 Mảnh ruộng 2

Đây là một mảnh ruộng có dạng hình thang, với điểm bắt đầu nằm gần trung điểm cạnh dưới.

5.2.1 Chiến thuật 1: Chia để trị + Đi theo hàng

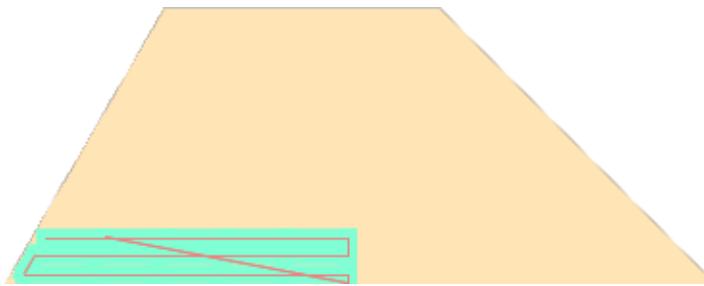
Chúng tôi sử dụng một chiến thuật tương tự với **Chiến thuật đi theo hàng** của mảnh ruộng 1. Mỗi khi drone hết thuốc hoặc hết pin thì drone sẽ quay lại điểm sạc.

Tuy nhiên do mảnh ruộng 2 có hình thang nên chúng tôi chia mảnh ruộng 2 thành 2 phần và làm với từng phần. Phần 1 là phần nằm ở bên trái của điểm sạ và phần 2 là phần nằm ở bên phải điểm sạ.



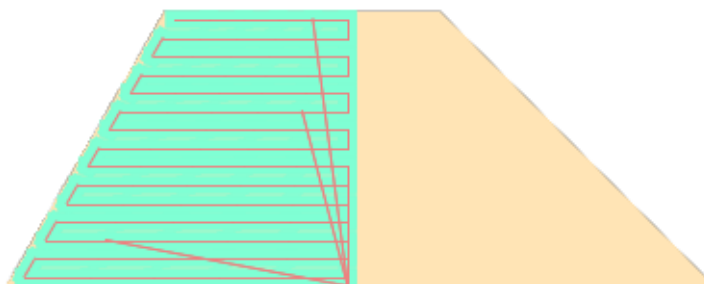
Hình 5.6: 2 phần của mảnh ruộng

Trong phần 1 drone sẽ bắt đầu phun thuốc từ điểm sạ rồi di chuyển qua các hàng lần lượt từ dưới lên trên.



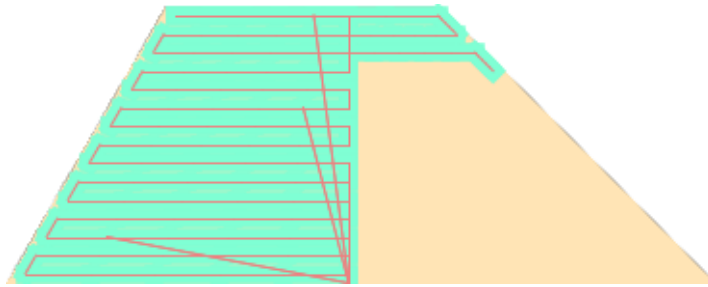
Hình 5.7: Drone đi theo từng hàng từ dưới lên trên ở phần 1

Sau khi kết thúc phần 1, drone đang ở vị trí trên cùng của mảnh ruộng. Trong phần 1, drone đã phải quay về điểm sạ tổng cộng 3 lần.



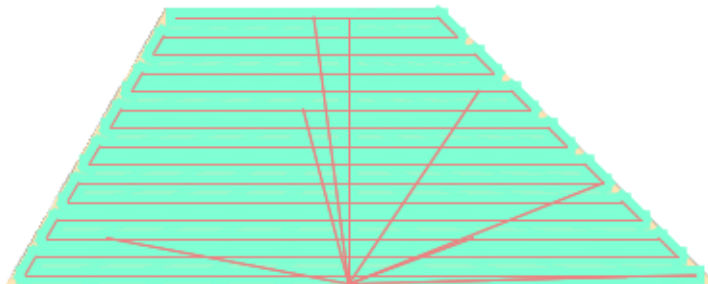
Hình 5.8: Drone sau khi kết thúc phần 1

Sau khi hoàn thành phần 1, drone đang ở vị trí trên cùng. Từ đây, drone sẽ di chuyển sang phần 2 và phun thuốc qua các hàng từ trên xuống dưới.



Hình 5.9: Drone đi theo từng hàng từ trên xuống dưới ở phần 2

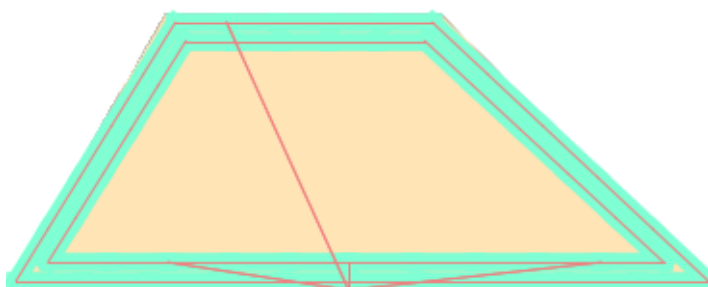
Sau khi hoàn thành phần 2 hay hết cả mảnh ruộng, drone quay trở về điểm sạc. Chiến thuật này cần **11895.125 giây** để thực hiện và tiêu thụ **96738.975 Wh**



Hình 5.10: Drone thành công phun toàn bộ ruộng

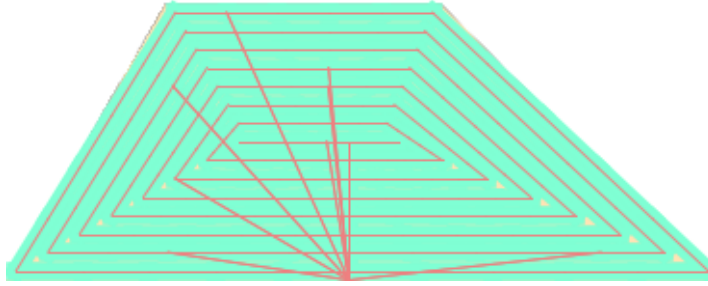
5.2.2 Chiến thuật 2: Vòng đồng tâm

Thuật toán cho drone đi xung quanh các cạnh của thửa ruộng và sau khi đi hết, thửa ruộng sẽ trở thành một hình thang có cạnh bé hơn và khi đấy chúng tôi sẽ lặp lại thuật toán cho đến khi drone phun hết mảnh ruộng.



Hình 5.11: Drone bắt đầu phun xung quanh mảnh ruộng

Sau khi hoàn thành phần 2 hay hết cả mảnh ruộng, drone quay trở về điểm sạc. Chiến thuật này cần **11837.729 giây** để thực hiện và tiêu thụ **95699.964 Wh**



Hình 5.12: Drone hoàn thành phun hết toàn bộ mảnh ruộng

Chúng tôi rút ra được kết luận: Đối với mảnh ruộng 2, cả 2 chiến thuật cần sử dụng lượng điện và thời gian gần bằng nhau.

5.3 Mảnh ruộng 3

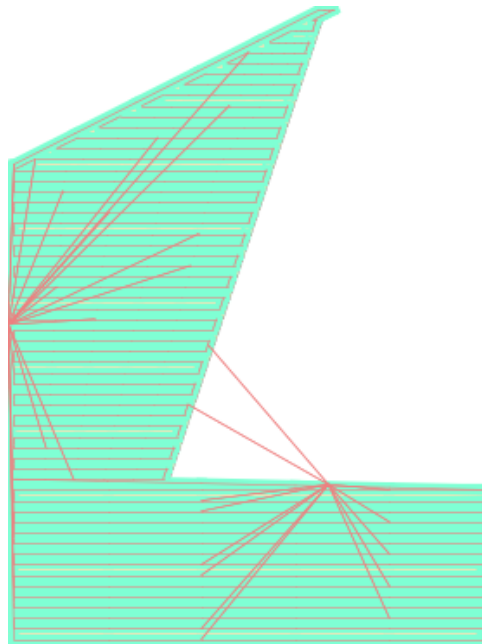
Đây là một mảnh ruộng kích thước lớn, bao gồm một hình chữ nhật $450m \times 150m$ ở phía dưới, và một hình tứ giác lớn ở phía trên.

Có 2 điểm sạc trong thửa ruộng này, một ở cạnh trên của hình chữ nhật và một ở cạnh trái của hình tứ giác trên.

5.3.1 Chiến thuật 1: Đi theo hàng

Chiến thuật này sẽ cho drone phun lần lượt từng hàng.

Drone sẽ xuất phát từ điểm sạc bên trái và di chuyển thẳng xuống phía dưới, sau đó dần dần đi theo các hàng lên trên.



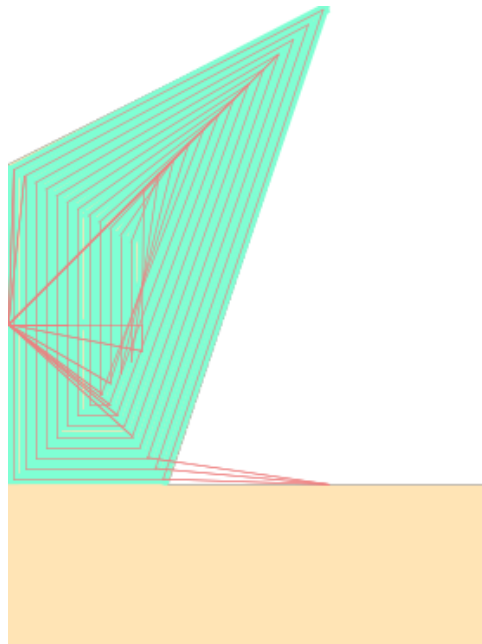
Hình 5.13: Drone hoàn thành việc phun thuốc

Chiến thuật này cần **42114.98 giây** để thực hiện, đồng thời cần tới **347151.94 Wh** để hoàn thành.

5.3.2 Chiến thuật 2: Chia để trị + Vòng đồng tâm

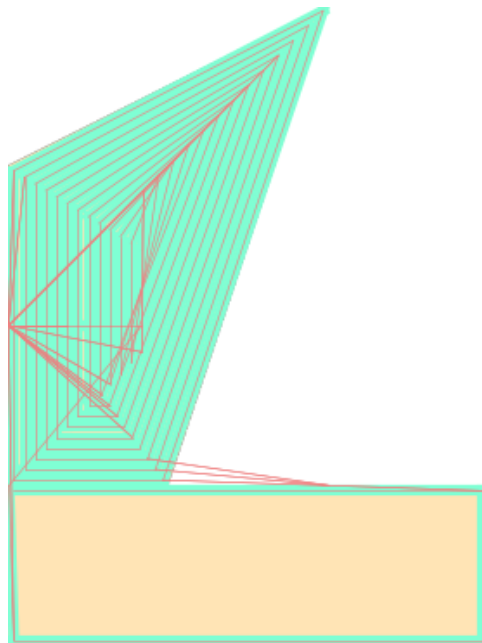
Chúng tôi chia thửa ruộng thành 2 phần như đã nói ở trên, sau đó đi vòng quanh từng phần. Mỗi khi drone cần quay về, ta sẽ chọn điểm gần nhất để quay về để tiết kiệm thời gian.

Chiến thuật này bắt đầu ở điểm bên cạnh trái, drone sẽ bắt đầu đi vòng quanh phần tứ giác phía trên. Tương tự như trên, drone sẽ đi đến điểm gần hơn để sạc.



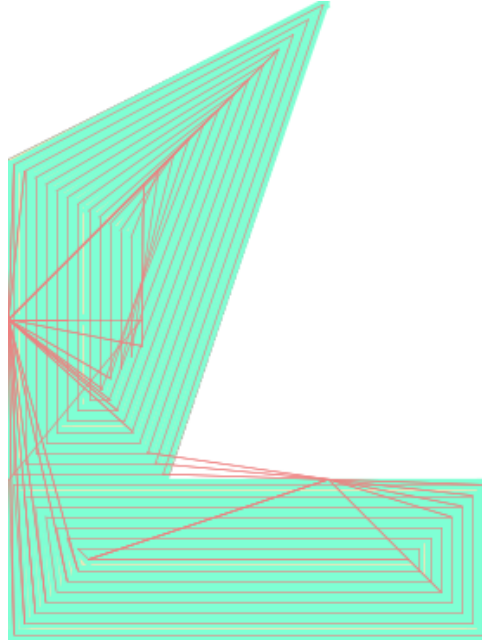
Hình 5.14: Drone đi vòng để vẽ hình tứ giác phía trên

Sau khi hoàn thành, drone di chuyển tới góc dưới bên phải để tiếp tục phun.



Hình 5.15: Drone bắt đầu đi xuống và đi vòng hình chữ nhật dưới

Thuật toán này cần **34587.58 giây** để hoàn thành và tiêu thụ **312824.34 Wh** trong quá trình phun.



Hình 5.16: Drone hoàn thành phun thuốc thửa ruộng

Chúng tôi rút ra được kết luận: Đối với mảnh ruộng 3, **chiến thuật chia để trị + vòng đồng tâm** sẽ cần thời gian và lượng điện ít hơn nhiều so với **chiến thuật đi theo hàng**.

5.4 Tổng kết

Trong cả 3 thửa ruộng thu được, chúng tôi thấy chiến thuật **chia để trị** quan trọng để chia nhỏ thửa ruộng và phun thuốc cho từng phần.

Sau đó, chúng tôi thấy rằng thuật toán **vòng đồng tâm** là chiến thuật tối ưu cả về mặt thời gian và năng lượng tiêu hao. Chúng tôi đã xét những chiến thuật di chuyển khác, tuy nhiên chúng không đạt được hiệu quả cao như 2 thuật toán được trình bày trong bản báo cáo này.

6 Phụ lục

6.1 Chương trình mô phỏng

Toàn bộ chương trình chạy mô phỏng của báo cáo này có thể được tìm thấy tại đường dẫn <https://1drv.ms/f/s!AlISTfgr2pIQj8891ajqMUTIsTu2PQ?e=eFuU4j>.

Sau đây là đoạn mã mô phỏng quá trình chạy của chúng tôi:

```

1 import os
2 import shutil
3 import math
4 import copy
5 import numpy as np
6 from collections import deque
7 from matplotlib import pyplot as plt
8 from matplotlib.lines import Line2D
9 Point = tuple[float, float]
10 class Board:
11     xp, yp = [0, -1, 0, 1], [-1, 0, 1, 0]
12     EPSILON = 1e-7
13     def __init__(self, vertices: tuple[Point], terminals: tuple[Point]):
14         self.vertices = vertices
15         self.terminals = terminals
16         assert len(self.vertices) > 2
17         self.size = 0
18         for Point in self.vertices:
19             self.size = max(self.size, Point[0], Point[1])
20         self.size = math.ceil(self.size)
21         self.board = [[False for _ in range(self.size)]
22                       for _ in range(self.size)]
23         self.__calculate_board()
24     def to_ndarray(self) -> np.ndarray:
25         COLOR_FALSE = [151, 153, 155]
26         COLOR_TRUE = [112, 224, 0]
27         self.np_board = [[[COLOR_FALSE, COLOR_TRUE][cell]
28                           for cell in line] for line in self.board]
29         self.np_board = np.array(self.np_board)
30         return self.np_board
31     def __crossed(self, cx: float, cy: float, nx: float, ny: float) -> bool:
32     def __counter_clockwise(a: Point, b: Point, c: Point) -> float:
33         return (a[0]-b[0])*c[1] + (b[0]-c[0])*a[1] + (c[0]-a[0])*b[1]
34     def __different_half_plane(x: Point, y: Point, a: Point, b: Point) -> bool:
35         return __counter_clockwise(x, a, b) * __counter_clockwise(y, a, b) < self.
36             EPSILON
37     def __intersected(c: Point, n: Point, a: Point, b: Point) -> bool:
38         return __different_half_plane(c, n, a, b) and __different_half_plane(a, b, c,
39             n)
40     for i in range(len(self.vertices)):
41         a, b = self.vertices[i-1], self.vertices[i]
42         if __intersected((cx, cy), (nx, ny), a, b):
43             return True
44     return False

```

```

43 def __calculate_board(self) -> None:
44     queue = deque()
45     queue.append((0, 0))
46     while len(queue):
47         cx, cy = queue.popleft()
48         for xa, ya in zip(self.xp, self.yp):
49             nx, ny = cx + xa, cy + ya
50             if self.__crossed(cx + .5, cy + .5, nx + .5, ny + .5):
51                 continue
52             if self.board[nx][ny]:
53                 continue
54             self.board[nx][ny] = 1
55             queue.append((nx, ny))
56 def to_image(self, ax: plt.Axes) -> None:
57     board = []
58     for y in range(self.size):
59         board.append([])
60         for x in range(self.size):
61             result = None
62             match self.board[x][y]:
63                 case 0:
64                     result = 0, 0, 0, 0
65                 case 1:
66                     result = 255, 228, 181, 255
67             board[-1].append(result)
68     ax.axis('off')
69     ax.imshow(board, origin="lower")
70 def __repr__(self):
71     s = ""
72     for y in range(self.size-1, -1, -1):
73         for x in range(self.size):
74             s += 'X' if self.board[x][y] else '-'
75             s += '\n'
76     return s
77 BOARD1 = (.0, .0), (100.0, .0), (100.0, 50.0), (.0, 50.0)
78 TERM1 = ((50, 0),)
79 BOARD2 = (.0, .0), (386.6025, .0), (236.6025, 150.0), (86.6025, 150.0)
80 TERM2 = ((186, 0),)
81 BOARD3 = (.0, .0), (450.0, .0), (450.0, 150.0), \
82     (150.0, 150.0), (300.0, 600.0), (.0, 450.0)
83 TERM3 = ((0, 300), (300, 150))
84 boards = {}
85 boards[1] = Board(BOARD1, TERM1)
86 boards[2] = Board(BOARD2, TERM2)
87 boards[3] = Board(BOARD3, TERM3)
88 class BatteryException(Exception):
89     pass
90 class Battery:
91     voltage = 110
92     def __init__(self, capacity: float):
93         self.capacity = capacity * self.voltage
94         self.remaining = self.capacity
95     def calculate(self, amount: float):
96         if self.remaining < amount:

```

```

97         raise BatteryException()
98         self.remaining -= amount
99 class ContainerException(Exception):
100     pass
101 class Container:
102     def __init__(self, battery: Battery, ct: float, capacity: float,
103                 remaining: float | None = None):
104         self.ct = ct
105         self.capacity = capacity
106         self.remaining = capacity if remaining is None else remaining
107         self.battery = battery
108     def calculate(self, seconds: float, used: float):
109         hours = seconds / 3600
110         self.battery.calculate(
111             self.ct * (self.remaining * 2 - used) / 2 * hours)
112         if self.remaining < used:
113             raise ContainerException
114         self.remaining -= used
115 class Pump:
116     def __init__(self, battery: Battery, container: Container,
117                 cp: float, max_spray_range: float, max_flow: float,
118                 fp: float | None = None, spray_range: float | None = None):
119         self.battery, self.container = battery, container
120         self.cp = cp
121         self.max_flow = max_flow
122         self.fp = self.max_flow if fp is None else fp
123         self.max_spray_range = max_spray_range
124         self.spray_range = self.max_spray_range if spray_range is None else spray_range
125     def change_fp(self, fp: float):
126         assert self.max_flow >= self.fp
127         self.fp = fp
128     def change_range(self, spray_range: float):
129         self.spray_range = spray_range
130     def calculate(self, seconds: float):
131         hours = seconds / 3600
132         self.battery.calculate(self.cp * self.fp * hours)
133         self.container.calculate(seconds, self.fp * hours)
134 class Drone:
135     def __init__(self, battery: Battery, pump: Pump, c: float, max_speed: float, v: float
136                 | None = None):
137         self.battery = battery
138         self.pump = pump
139         self.c = c
140         self.max_speed = max_speed
141         self.v = max_speed if v is None else v
142     def change_speed(self, v: float):
143         assert self.max_speed >= v
144         self.v = v
145     def change_speed_based_on_pump(self, concn: float):
146         assert self.pump.fp != 0
147         assert self.pump.spray_range != 0
148         # Convert to L/m^2
149         concn /= 1e4
150         new_speed = self.pump.fp / (concn * self.pump.spray_range)

```

```

150     # Convert to m/s
151     new_speed /= 3600
152     self.change_speed(new_speed)
153     def calculate(self, seconds: float):
154         hours = seconds / 3600
155         self.pump.calculate(seconds)
156         self.battery.calculate(self.c * hours)
157     def normal_working(self, concen: float):
158         self.pump.change_fp(self.pump.max_flow)
159         self.change_speed_based_on_pump(concen)
160 # T40 Constants
161 DEFAULT_SPRAY_RANGE = 10
162 MAX_FLOW = 720
163 battery = Battery(30)
164 container = Container(battery, ct=857.14, capacity=40)
165 pump = Pump(battery, container, cp=0.42,
166             max_spray_range=DEFAULT_SPRAY_RANGE, max_flow=MAX_FLOW)
167 default_drone = Drone(battery, pump, c=11000, max_speed=10)
168 def scale_vector(vector: Point, dist: float):
169     mult = (vector[0] ** 2 + vector[1] ** 2) ** 0.5 / dist
170     if abs(mult) > 1e-9:
171         vector = (vector[0] / mult, vector[1] / mult)
172     return vector
173 # Reference: https://stackoverflow.com/questions/19394505/expand-the-line-with-specified-width-in-data-unit/42972469
174 class LineDataUnits(Line2D):
175     def __init__(self, *args, **kwargs):
176         _lw_data = kwargs.pop("lw", 1)
177         super().__init__(*args, **kwargs)
178         self._lw_data = _lw_data
179     def _get_lw(self):
180         if self.axes is not None:
181             ppd = 72./self.axes.figure.dpi
182             trans = self.axes.transData.transform
183             return ((trans((1, self._lw_data))-trans((0, 0)))*ppd)[1]
184         else:
185             return 1
186     def _set_lw(self, lw):
187         self._lw_data = lw
188     _linewidth = property(_get_lw, _set_lw)
189 def draw_spray_range(a: Point, b: Point, ax: plt.Axes, **kwargs):
190     line = LineDataUnits([a[0], b[0]], [a[1], b[1]], **kwargs)
191     ax.add_line(line)
192 def distance(a: Point, b: Point) -> float:
193     return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** .5
194 class Run:
195     def __init__(self, board: Board, drone: Drone,
196                 position: Point, run_on_critical):
197         self.time_spent = 0
198         self.battery_spent = 0
199         self.board, self.drone = board, drone
200         self.movement = [0, 0]
201         self.position = position
202         self.run_on_critical = run_on_critical

```

```

203     self.fig, self.ax = plt.subplots()
204     self.board.to_image(self.ax)
205     # Vector auto-scaled to current velocity
206     def set_movement(self, vector: tuple[float, float]):
207         self.movement = scale_vector(vector, self.drone.v)
208     # Vector auto-scaled to current velocity
209     def point_to(self, position: Point):
210         vector = position[0] - self.position[0], position[1] - self.position[1]
211         self.set_movement(vector)
212     def new_position(self, seconds: float) -> Point:
213         position = (self.position[0] + seconds * self.movement[0],
214                    self.position[1] + seconds * self.movement[1])
215         return position
216     def check_critical(self):
217         min_distance = 1e18
218         for term in self.board.terminals:
219             min_distance = min(min_distance, distance(term, self.position))
220         time_needed = min_distance / self.drone.max_speed
221         try:
222             cpy = copy.deepcopy(self.drone)
223             cpy.change_speed(cpy.max_speed)
224             cpy.pump.change_fp(0)
225             cpy.calculate(time_needed)
226         except BatteryException:
227             self.run_on_critical(self)
228     def calculate(self, seconds: float):
229         prev_battery = self.drone.battery.remaining
230         self.drone.calculate(seconds)
231         new_battery = self.drone.battery.remaining
232         self.time_spent += seconds
233         self.battery_spent += prev_battery - new_battery
234         self.check_critical()
235         new_position = self.new_position(seconds)
236         # Draw rectangle from position to new_position
237         # Width = self.drone.pump.spray_range
238         draw_spray_range(self.position, new_position, self.ax,
239                          lw=self.drone.pump.spray_range, color='aquamarine', zorder=4)
240         draw_spray_range(self.position, new_position, self.ax,
241                          lw=1, color='lightcoral', zorder=5)
242         self.position = new_position
243     # Speed must be set first
244     def go_to(self, position: Point):
245         cpy = copy.deepcopy(self)
246         try:
247             dist = distance(self.position, position)
248             time_needed = dist / self.drone.v
249             self.point_to(position)
250             self.calculate(time_needed)
251         except Exception as e:
252             self = cpy
253             raise e
254 def get_board1_sol1(run: Run) -> tuple[list[tuple[float, float]], str]:
255     DRONE_PATH = []
256     SPRAY_RANGE = 10

```

```

257     for i in range(0, 3):
258         DRONE_PATH += [(50, SPRAY_RANGE / 2 + SPRAY_RANGE * i),
259                        (100 - SPRAY_RANGE / 2 - SPRAY_RANGE *
260                         i, SPRAY_RANGE / 2 + SPRAY_RANGE * i),
261                        (100 - SPRAY_RANGE / 2 - SPRAY_RANGE * i,
262                         50 - SPRAY_RANGE / 2 - SPRAY_RANGE * i),
263                        (SPRAY_RANGE / 2 + SPRAY_RANGE * i,
264                         50 - SPRAY_RANGE / 2 - SPRAY_RANGE * i),
265                        (SPRAY_RANGE / 2 + SPRAY_RANGE * i,
266                         SPRAY_RANGE / 2 + SPRAY_RANGE * i),
267                        (50, SPRAY_RANGE / 2 + SPRAY_RANGE*i)]
268     return DRONE_PATH, "board1_sol1"
269 def get_board1_sol2(run: Run) -> tuple[list[tuple[float, float]], str]:
270     SPRAY_RANGE = 50/6
271     DRONE_PATH = [(50, SPRAY_RANGE / 2)]
272     leftx, rightx = SPRAY_RANGE * 1.5, 100 - SPRAY_RANGE/2
273     for i in range(0, 6, 2):
274         DRONE_PATH.append((rightx, SPRAY_RANGE*(i + 0.5)))
275         DRONE_PATH.append((rightx, SPRAY_RANGE*((i+1) + 0.5)))
276         DRONE_PATH.append((leftx, SPRAY_RANGE*((i+1) + 0.5)))
277         DRONE_PATH.append((leftx, SPRAY_RANGE*((i+2) + 0.5)))
278     DRONE_PATH.pop()
279     DRONE_PATH.pop()
280     DRONE_PATH.append((SPRAY_RANGE/2, 50 - SPRAY_RANGE/2))
281     DRONE_PATH.append((SPRAY_RANGE/2, SPRAY_RANGE/2))
282     DRONE_PATH.append((50, SPRAY_RANGE/2))
283     return DRONE_PATH, "board1_sol2"
284 def get_board2_sol1(run: Run) -> tuple[list[tuple[float, float]], str]:
285     DRONE_PATH = []
286     SPRAY_RANGE = 10
287     for i in range(8):
288         DRONE_PATH.append((run.board.terminals[0][0],
289                             run.board.vertices[0][1] + SPRAY_RANGE * (i + 0.5)))
290         DRONE_PATH.append((run.board.vertices[1][0] - SPRAY_RANGE * (i * 1/math.tan(math.
291             pi*(1/8)) + 0.5),
292                             run.board.vertices[1][1] + SPRAY_RANGE * (i + 0.5)))
293         DRONE_PATH.append((run.board.vertices[2][0] - SPRAY_RANGE * (i * 1/math.tan(math.
294             pi*(3/8)) + 0.5),
295                             run.board.vertices[2][1] - SPRAY_RANGE * (i + 0.5)))
296         DRONE_PATH.append((run.board.vertices[3][0] + SPRAY_RANGE * (i * 1/math.tan(math.
297             pi/3) + 0.5),
298                             run.board.vertices[3][1] - SPRAY_RANGE * (i + 0.5)))
299         DRONE_PATH.append((run.board.vertices[0][0] + SPRAY_RANGE * (i * 1/math.tan(math.
300             pi/6) + 0.5),
301                             run.board.vertices[0][1] + SPRAY_RANGE * (i + 0.5)))
302         DRONE_PATH.append((run.board.terminals[0][0],
303                             run.board.vertices[0][1] + SPRAY_RANGE * (i + 0.5)))
304     return DRONE_PATH, "board2_sol1"
305 def get_board2_sol2(run: Run) -> tuple[list[tuple[float, float]], str]:
306     DRONE_PATH = []
307     SPRAY_RANGE = 10
308     for i in range(0, 16, 2):
309         DRONE_PATH.append((run.board.terminals[0][0], SPRAY_RANGE * (i + 0.5)))
310         DRONE_PATH.append(

```

```

307         (SPRAY_RANGE*(i * 0.580123 + 1), SPRAY_RANGE * (i + 0.5)))
308     DRONE_PATH.append((SPRAY_RANGE*((i+1) * 0.580123 + 1),
309                        SPRAY_RANGE * ((i+1) + 0.5)))
310     DRONE_PATH.append(
311         (run.board.terminals[0][0], SPRAY_RANGE * ((i+1) + 0.5)))
312     DRONE_PATH.pop()
313     DRONE_PATH.pop()
314     crrx, crry = run.board.vertices[2][0] - \
315         SPRAY_RANGE / 2 + 3, DRONE_PATH[-1][1]
316     for i in range(0, 16, 2):
317         DRONE_PATH.append((crrx + SPRAY_RANGE * i, crry - SPRAY_RANGE * i))
318         DRONE_PATH.append((crrx + SPRAY_RANGE * (i+1),
319                            crry - SPRAY_RANGE * (i+1)))
320         DRONE_PATH.append(
321             (run.board.terminals[0][0], crry - SPRAY_RANGE * (i+1)))
322         DRONE_PATH.append(
323             (run.board.terminals[0][0], crry - SPRAY_RANGE * (i+2)))
324     DRONE_PATH.pop()
325     DRONE_PATH.pop()
326     DRONE_PATH.pop()
327     return DRONE_PATH, "board2_sol2"
328 def get_board3_sol1(run: Run) -> tuple[list[tuple[float, float]], str]:
329     SPRAY_RANGE = 10
330     DRONE_PATH = []
331     polygon = [(0.0, 150.0), (150.0, 150.0), (300.0, 600.0), (0.0, 450.0)]
332     for i in range(13):
333         DRONE_PATH.append((run.board.terminals[0][0] + SPRAY_RANGE * (i + 0.5),
334                            run.board.terminals[0][1]))
335         DRONE_PATH.append((polygon[0][0] + SPRAY_RANGE * (i + 0.5),
336                            polygon[0][1] + SPRAY_RANGE * (i + 0.5)))
337         if polygon[1][0] - SPRAY_RANGE * (i * 1/math.tan(math.radians(108.43/2)) + 0.5) >
338             polygon[0][0] + SPRAY_RANGE * (i + 0.5):
339             DRONE_PATH.append((polygon[1][0] - SPRAY_RANGE * (i * 1/math.tan(math.radians
340                                     (108.43/2)) + 0.5),
341                                polygon[1][1] + SPRAY_RANGE * (i + 0.5)))
342         DRONE_PATH.append((polygon[2][0] - SPRAY_RANGE * (i*1.4 + 0.5),
343                            polygon[2][1] - SPRAY_RANGE * (i*1.4 + 0.5)))
344         DRONE_PATH.append((polygon[3][0] + SPRAY_RANGE * (i + 0.5),
345                            polygon[3][1] - SPRAY_RANGE * (i * 1/math.tan(math.radians
346                                     (116.65/2)) + 0.5)))
347         DRONE_PATH.append((run.board.terminals[0][0] + SPRAY_RANGE * (i + 0.5),
348                            run.board.terminals[0][1]))
349     DRONE_PATH.append((polygon[0][0]+0.5, polygon[0][1]))
350     polygon = [(0.0, 0.0), (450.0, 0.0), (450.0, 150.0), (0.0, 150.0)]
351     for i in range(8):
352         DRONE_PATH.append(
353             (polygon[0][0] + SPRAY_RANGE*(i + 0.5), polygon[0][1] + SPRAY_RANGE*(i+0.5)))
354         DRONE_PATH.append(
355             (polygon[1][0] - SPRAY_RANGE*(i + 0.5), polygon[1][1] + SPRAY_RANGE*(i+0.5)))
356         DRONE_PATH.append(
357             (polygon[2][0] - SPRAY_RANGE*(i + 0.5), polygon[2][1] - SPRAY_RANGE*(i+0.5)))
358         DRONE_PATH.append(
359             (polygon[3][0] + SPRAY_RANGE*(i + 0.5), polygon[3][1] - SPRAY_RANGE*(i+0.5)))
360     return DRONE_PATH, "board3_sol1"

```

```

358 def get_board3_sol2(run: Run) -> tuple[list[tuple[float, float]], str]:
359     SPRAY_RANGE = 10
360     DRONE_PATH = []
361     polygon = [(0.0, 150.0), (150.0, 150.0), (300.0, 600.0), (.0, 450.0)]
362     lbx, rbx = SPRAY_RANGE/2, 450 - SPRAY_RANGE/2
363     for i in range(0, 16, 2):
364         DRONE_PATH.append((lbx, SPRAY_RANGE*(i + 0.5)))
365         DRONE_PATH.append((rbx, SPRAY_RANGE*(i + 0.5)))
366         DRONE_PATH.append((rbx, SPRAY_RANGE*((i+1) + 0.5)))
367         DRONE_PATH.append((lbx, SPRAY_RANGE*((i+1) + 0.5)))
368     DRONE_PATH.pop()
369     DRONE_PATH.pop()
370     lbx = SPRAY_RANGE/2
371     rbx = 150 - SPRAY_RANGE/2
372     sty = 150 + SPRAY_RANGE/2
373     for i in range(0, 30, 2):
374         DRONE_PATH.append((lbx, sty + SPRAY_RANGE*i))
375         DRONE_PATH.append(
376             (rbx + SPRAY_RANGE*(i * 1 / math.tan(math.radians(71.56))), sty + SPRAY_RANGE
377              * i))
378         DRONE_PATH.append((rbx + SPRAY_RANGE*((i+1) * 1 /
379             math.tan(math.radians(71.56))), sty + SPRAY_RANGE*(i+1)))
380         DRONE_PATH.append((lbx, sty + SPRAY_RANGE*(i+1)))
381     sty = 450 + SPRAY_RANGE/2
382     rbx = 250 - SPRAY_RANGE/2
383     for i in range(0, 16, 2):
384         DRONE_PATH.append((lbx + SPRAY_RANGE*((i+1) * 1 /
385             math.tan(math.radians(26.56))), sty + SPRAY_RANGE*i))
386         DRONE_PATH.append(
387             (rbx + SPRAY_RANGE*(i * 1 / math.tan(math.radians(71.56))), sty + SPRAY_RANGE
388              * i))
389         DRONE_PATH.append((rbx + SPRAY_RANGE*((i+1) * 1 /
390             math.tan(math.radians(71.56))), sty + SPRAY_RANGE*(i+1)))
391         DRONE_PATH.append((lbx + SPRAY_RANGE*((i+2) * 1 /
392             math.tan(math.radians(26.56))), sty + SPRAY_RANGE*(i+1)))
393     DRONE_PATH.pop()
394     DRONE_PATH.pop()
395     DRONE_PATH.append((SPRAY_RANGE/2, 450))
396     return DRONE_PATH, "board3_sol2"
397
398 # Constants
399 COLOR_WHITE = (255, 255, 255)
400 SPRAY_RANGE = 10
401 def fine_grain_path(start: tuple[float, float], drone_path: list[tuple[float, float]]) ->
402     list[tuple[float, float]]:
403     result = []
404     for point in drone_path:
405         segment = 5
406         vector = (point[0] - start[0]) / \
407             segment, (point[1] - start[1]) / segment
408         for i in range(segment):
409             result.append((start[0] + vector[0] * (i+1),
410                 start[1] + vector[1] * (i+1)))
411         start = point
412     return result

```



```

409 def critical(run: Run):
410     run.drone.pump.change_fp(0)
411     run.drone.pump.change_range(0)
412     run.drone.change_speed(run.drone.max_speed)
413     try:
414         if len(run.board.terminals) < 2:
415             run.go_to(run.board.terminals[0])
416         else:
417             if distance(run.board.terminals[0], run.position) < distance(run.board.
418                 terminals[1], run.position):
419                 run.go_to(run.board.terminals[0])
420             else:
421                 run.go_to(run.board.terminals[1])
422     except BatteryException:
423         pass
424     print("New drone")
425     run.drone = copy.deepcopy(default_drone)
426 board = boards[1]
427 run = Run(board, copy.deepcopy(default_drone), board.terminals[0], critical)
428 run.drone.pump.change_range(SPRAY_RANGE)
429 DRONE_PATH, NAME = get_board1_sol1(run)
430 DRONE_PATH = fine_grain_path(
431     run.position, DRONE_PATH + [run.board.terminals[0]])
432 # Reference: https://stackoverflow.com/questions/43096972/how-can-i-render-a-matplotlib-axes-object-to-an-image-as-a-numpy-array
433 def save_ax(ax: plt.Axes, filename: str, **kwargs):
434     ax.axis("off")
435     ax.figure.canvas.draw()
436     trans = ax.figure.dpi_scale_trans.inverted()
437     bbox = ax.bbox.transformed(trans)
438     plt.savefig(filename, dpi="figure", bbox_inches=bbox, **kwargs)
439 try:
440     shutil.rmtree(f"images/{NAME}")
441     os.makedirs(f"images/{NAME}")
442 except Exception as e:
443     print(e)
444 CONCENTRATION = 75
445 for idx, point in enumerate(DRONE_PATH):
446     run.drone.change_speed_based_on_pump(CONCENTRATION)
447     try:
448         run.go_to(point)
449     except ContainerException:
450         position = run.position
451         critical(run)
452         run.drone.pump.change_fp(0)
453         run.drone.pump.change_range(0)
454         run.go_to(position)
455         run.drone.pump.change_fp(run.drone.pump.max_flow)
456         run.drone.pump.change_range(run.drone.pump.max_spray_range)
457         run.go_to(point)
458     print(run.drone.battery.remaining, run.time_spent,
459           run.battery_spent, run.drone.pump.container.remaining)
460 save_ax(run.ax, f"images/{NAME}/{idx}.png")
461 print(NAME)

```

```
461 print(f"Concentration: {CONCENTRATION}L/ha")
462 print(f"Time spent: {run.time_spent*(500/CONCENTRATION)}s")
463 print(f"Energy consumed: {run.battery_spent*(500/CONCENTRATION)}Wh")
464 def export(FOLDER_NAME: str):
465     import glob
466     from pathlib import Path
467     import imageio
468     FOLDER_NAME = "images\\boardx_soly"
469     def srt(a: str):
470         return int(Path(a).stem)
471     image_names = glob.glob(f"{FOLDER_NAME}\\*.png")
472     image_names = sorted(image_names, key=srt)
473     images = []
474     for name in image_names:
475         images.append(imageio.imread(name))
476     imageio.mimsave(f"{FOLDER_NAME}\\result.gif", images)
```

6.2 Tài liệu tham khảo

Hướng dẫn sử dụng drone DJI Agras T40: https://dl.djicdn.com/downloads/t40_t20p/20221212/T40_T20P_User_Manual_v1.2_EN.pdf

Chương trình chạy của nhóm: <https://1drv.ms/f/s!AlISTfgr2pIQj8891ajqMUTIsTu2PQ?e=eFuU4j>